CookMate

An Interactive Cooking Copilot

LT2319 Dialogue Systems 2 (2026)

Eugene Wong

Agenda

1. Problem
2. Pre-existing API
3. Data Collection & Distillation
4. System Architecture & Implementation
5. Sample Dialogue
6. Future Work
7. Live Demo

Cooking as a Dialogue Domain

- Cooking is inherently **hands-busy and eyes-busy**
- **Noisy, hostile acoustic environment**:
  - Sizzling pans, running water, and extractor fans compete with voice input
- **Asymmetric pacing & long pauses**:
  - Multi-minute gaps between turns (chopping, simmering) while state must persist
- **Frequent interruptions & contextual inquiries**:
  - **Clarifications**: *"Wait, how many tablespoons of butter?"*
  - **Substitutions**: *"I don't have heavy cream, what can I use?"*
  - **Technique help**: *"What does al dente actually look like?"*
- **Concurrent events**:
  - Independent timers running simultaneously while the user preps other ingredients

Why Pure LLM Chatbots Fail in the Kitchen

State Amnesia

- Loses track of completed and remaining steps across long sessions
- Cannot guarantee strict, step-by-step progress through a recipe

Hallucination Risks

- Inventing missing measurements or altering cooking steps
- Presents real culinary failure and safety hazards

Poor Voice Ergonomics

- Spoken dialogue demands snappy responses
- LLMs fail on both: high latency and excessive length

No Grounded Action

- Cannot maintain set-and-forget timers in background tabs
- Cannot proactively alert you when cooking time is up

TheMealDB: Our Recipe Data Source

- **Open Community Database**: Around 300 recipes from diverse web sources
- **Inconsistent by Nature**:
  - Mixed measurements (cups, grams, spoons, or no units at all)
  - Varying writing styles with typos and informal phrasing
  - Mismatched items (ingredients mentioned in text but omitted from the list)
- **Rigid Structure**: Fixed 20 ingredient slots
- **Unstructured Text**: Instructions saved as one long block rather than separate steps
- **Zero Cooking Knowledge**: No built-in cooking times, techniques, or substitutes

What the Raw Recipe Data Looks Like


```
{
  "idMeal": "52772",
  "strMeal": "Teriyaki Chicken Casserole",
  "strCategory": "Chicken",
  "strIngredient1": "soy sauce",
  "strMeasure1": "3/4 cup",
  "strIngredient2": "water",
  "strMeasure2": "1/2 cup",
  "...": "...",
  "strIngredient20": "",
  "strMeasure20": "",
  "strInstructions": "Preheat oven to 350° F. Spray pan with cooking spray.
}
```

Recipe Normalization & Refinement

1. Code Cleanup

- Pairs ingredient slots with measurements
- Splits text blocks into single sentences
- Fast rule-based fallback without LLM

2. LLM Refinement

- Expands abbreviations into spoken words (`tbsp` → `tablespoon`)
- Adds forgotten basics to the ingredient list (oil, salt, water)
- Breaks compound tasks into short, single-action steps (< 25 words)

- Refined recipes are cached in browser storage to avoid repeated LLM calls

How People Talk While Cooking

- **Simulated Dialogues**:
  - **8 sample sessions, 102 spoken turns** between a home cook and an assistant
  - Studied natural speech patterns to guide system design before writing code
- **Common Speech Patterns Observed**:
  - **Vague words (*"it"*, *"that"*)**: Cooks say *"Can I add it now?"* or *"Would that work?"* without naming the ingredient
  - **Pacing requests**: Asking to pause (*"Hold on"*) or repeat (*"Say that again"*) without advancing the recipe
  - **Mid-step questions**: Asking about missing ingredients, measurements, or cooking techniques while working

What the Assistant Needs to Understand

Step Navigation

- `NEXT_STEP`, `PREVIOUS_STEP`
- `REPEAT_STEP`, `GOTO_STEP`
- `SKIP_PREP` (skip to cooking)

Cooking Questions

- `QUERY_INGREDIENT` (amounts)
- `QUERY_STEP_DETAIL` (step help)
- `INQUIRE_SUBSTITUTION` (swaps)
- `INQUIRE_TECHNIQUE` (methods)

Timers

- `SET_TIMER` (duration & label)
- `CHECK_TIMER` (time left)
- `CANCEL_TIMER`

Search & Basic Chat

- `SEARCH_RECIPE`, `SELECT_RECIPE`
- `AFFIRM`, `DENY` (yes / no)
- `GREETING`, `HELP`, `UNKNOWN` (repair)

- **Key Details Extracted (** `Entities` **):** `ingredient`, `stepNumber`, `durationSeconds`, `timerLabel`, `query`

Architecture Overview

Voice Interface

- **Tech**: `openWakeWord` + `Azure Speech`
- **Role**: Wake word trigger, ASR and TTS

NLU

- **Tech**: `Groq` with `openai/gpt-oss-20b`
- **Role**: Turns spoken words into clear cooking actions and commands

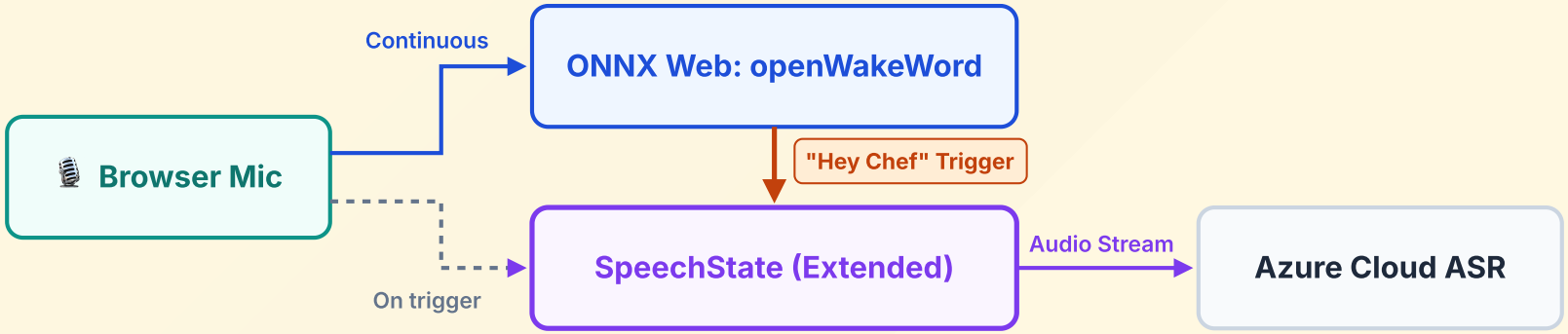
Orchestration & Runtime

- **Tech**: `SpeechState` + `XState` + `Web Worker`
- **Role**: Tracks cooking progress step-by-step and runs background timers

Culinary Knowledge & RAG

- **Tech**: `TheMealDB` + `Qdrant` + `Transformers.js`
- **Role**: Finds recipes and answers cooking questions with short, spoken advice

Hands-Free Audio Pipeline



- **Why local wake word?** Avoids a permanent open mic where random kitchen noise or side conversations trigger CookMate by mistake.
- **Why extend SpeechState?** CookMate was already capturing the mic for `openWakeWord`, so we extended `SpeechState` to accept that existing stream rather than fighting over the microphone.
- **Eyes-free design**: Subtle audio chimes confirm when the assistant wakes and sleeps, so the cook never has to look at the screen.

Dialogue Flow & Cooking Policy (XState)

Paced Ingredient Prep

- **No info overload**: Chunks ingredient lists into bite-sized pairs (2 at a time)
- **Explicit pacing**: Waits for *"got them"* before introducing the next batch

Flexible Step Navigation

- **Sequential flow**: Steps advance one by one with *"next"*
- **Repetitions & jumps**: Handles *"say that again"* or *"jump to step 4"* safely

Mid-Step Interruptions

- **Ask anytime**: Inquire about substitutions or measurements mid-step
- **Instant resume**: Answers the question and returns to the active step

Proactive Timer Prompts

- **Context-aware**: Detects durations mentioned in step text
- **Proactive offer**: Asks *"Want me to set a 5-minute timer?"* so the cook doesn't have to calculate

Concurrent Kitchen Timers

The "Sleeping Screen" Problem

- **Kitchen reality**: Cooks step away, screens dim, or tabs get backgrounded
- **The danger**: Browsers aggressively pause background apps to save battery—risking burnt food
- **Our fix**: A dedicated background thread (`Web Worker`) keeps counting down reliably

Concurrent Dialogue & Priority Alerts

- **Parallel tracking**: Runs multiple independent timers while you continue prepping
- **Non-destructive interrupt**: Timer alarms chime and speak over current dialogue without losing recipe progress
- **Conversational checks**: Cooks can ask *"how much time left on the pasta?"* at any time

Spoken NLU: Turning Speech into Actions

- Powered by `openai/gpt-oss-20b` on Groq for near-instant spoken understanding

Structured Cooking Commands

- **From speech to actions**: Translates natural speech into clear, typed commands
- **Zero guessing**: Enforces strict JSON schemas for navigation, timers, and questions
- **Spoken speed**: Responds near-instantly so conversation feels fluid

State-Conditioned Understanding

- **Resolves vague speech**: Uses current dialogue state to interpret ambiguous words:
  - In *Recipe Search*: *"the second one"* → `SELECT_RECIPE`
  - In *Ingredient Prep*: *"ready"* → `NEXT_STEP`
  - In *Cooking Step*: *"how much butter?"* → `QUERY_INGREDIENT`

Culinary Knowledge & Grounded Answers

Grounded Knowledge (RAG)

- **Local embeddings**: Runs `Transformers.js` in-browser to embed user queries
- **Verified search**: Matches queries against indexed culinary guides in `Qdrant`
- **Curated categories**: Targeted lookup for ingredient *substitutions* and *cooking techniques*

Spoken Voice Budget (< 25 Words)

- **Grounded synthesis**: Combines retrieved facts with recipe context using `Groq`
- **Strict voice limit**: Enforces a strict < 25-word limit so spoken advice never drags on

End-to-End Dialogue Walkthrough

Phase 1: Discovery & Paced Prep

- **User:** *"I want to make Beef Noodles"* → `[SEARCH_RECIPE]` *(hydrates & refines recipe)*
- **CookMate:** *"Found Beef Noodles. Ready to gather ingredients?"*
- **User:** *"Yes, ready"* → `[AFFIRM]`
- **CookMate:** *"First, gather 800 grams beef, 2 onions, and 2 carrots." (Batch 1 of 2)*
- **User:** *"Got them"* → `[AFFIRM]`

Phase 2: Cooking, RAG & Timers

- **CookMate:** *"Step 2: Heat 2 tablespoons of olive oil in a large pot."*
- **User:** *"Can I use butter instead?"* → `[INQUIRE_SUBSTITUTION]`
- **CookMate:** *"Yes, butter works well, but keep heat medium so it doesn't burn."*
- **User:** *"What's next?"* → `[NEXT_STEP]`
- **CookMate:** *"Step 3: Sear the beef for 5 minutes. Would you like me to start a timer for 5 minutes?"*
- **User:** *"Yes please."* → `[SET_TIMER]`

Future Improvements

Barge-In

- **Current status**: Enabling speech interruption caused erratic behavior between TTS and ASR, so it was disabled to ensure demo stability
- **Next step**: Troubleshoot and stabilize barge-in so cooks can interrupt or say *"stop"* mid-speech

Dynamic Portion Scaling

- **Current status**: Recipe APIs provide fixed ingredient quantities for rigid serving sizes
- **Next step**: Conversational math on the fly (*"cook for 6 people"*, *"how many cups is 200g?"*) without losing step progress

Persistent Dietary Memory

- **Current status**: Assistant starts fresh every time, forgetting user habits between sessions
- **Next step**: Remember preferences across visits (*"remember I'm lactose intolerant"*, *"always swap cilantro for parsley"*)

Demo